CAD_AI

Text-to-FeatureScript Custom Feature Library

FeatureScript Submission Packet

Author	noobifyLol
Repository	https://github.com/noobifyLol/CAD_AI
Language / version	Onshape FeatureScript 2931 (std library 2931.0)
Submission contents	24 parametric custom features (one Feature Studio each) plus the AI generator that produces new ones from plain English
Validation	24 / 24 features pass the project's strict FeatureScript validator; 20 / 20 run without errors in a live Onshape Part Studio
Date	October 09, 2026

1. Summary

CAD_AI turns a plain-English request such as “create a wheel with 6 spokes” into Onshape FeatureScript that adds editable, parametric geometry to a Part Studio. This packet submits the project's library of 24 custom features. Every feature is a complete Feature Studio file with one exported `defineFeature` and a dialog of editable parameters (lengths, counts, an optional sketch plane). The generator uses these features as its reference set and as a guaranteed fallback, so they are the project's core FeatureScript.

What makes the features useful

- Fully parametric. Every size is a dialog parameter with sensible bounds and defaults. Nothing is hard-coded, so a part can be resized after it is placed.
- Placeable. Each feature takes an optional Plane pick. If you leave it empty, the part is built on the world Top plane.
- Real decomposition. Everyday objects are built from their components (pen = barrel + tip + clip, wheel = rim + hub + patterned spokes) rather than one stretched primitive.
- Idiomatic FeatureScript. The features use standard library operations only (`opExtrude`, `opRevolve`, `opLoft`, `opSweep`, `opShell`, `opPattern`, `opBoolean`, `opFillet`, `opChamfer`).

2. Feature catalog

Each row is one custom feature. The Feature name is what appears in the Onshape toolbar and the Export is the FeatureScript symbol.

#	Feature name	Export	Key operations	Inputs	Static	Onshape
1	**Plate With Mounting Holes**	`plateWithMountingHoles`	opExtrude, opBoolean SUBTRACTION	6	PASS	PASS
2	**Cylinder With Bore**	`cylinderWithBore`	opExtrude	4	PASS	PASS
3	**Open Top Box**	`openTopBox`	opExtrude, opShell	5	PASS	PASS
4	**Cup With Handle**	`cupWithHandle`	opExtrude, opShell, opBoolean UNION	6	PASS	PASS
5	**Revolved Vase**	`revolvedVase`	opRevolve	5	PASS	PASS
6	**Square To Round Transition**	`squareToRoundTransition`	opLoft	4	PASS	PASS
7	**Swept Curved Tube**	`sweptCurvedTube`	opSweep	4	PASS	PASS
8	**L Bracket With Fillet And Chamfer**	`lBracketFilletChamfer`	opExtrude, opFillet	6	PASS	PASS
9	**Bolt Circle Flange**	`boltCircleFlange`	opExtrude, for loop	7	PASS	PASS
10	**Base With Corner Posts**	`baseWithCornerPosts`	opExtrude, opBoolean UNION	6	PASS	PASS
11	**Stepped Shaft With Chamfer**	`steppedShaftWithChamfer`	opExtrude, opBoolean UNION, opChamfer	6	PASS	PASS
12	**Disc With Patterned Bosses**	`discWithPatternedBosses`	opExtrude, opPattern, rotationAround, opBoolean UNION	7	PASS	PASS
13	**Heatsink With Linear Fins**	`heatsinkWithLinearFins`	opExtrude, opPattern, transform, opBoolean UNION	8	PASS	PASS
14	**Pen With Tip And Clip**	`penWithTipAndClip`	opExtrude, opLoft, opBoolean UNION	6	PASS	PASS
15	**Bottle With Neck**	`bottleWithNeck`	opRevolve, opShell	6	PASS	PASS
16	**Hex Bolt**	`hexBolt`	skRegularPolygon, opExtrude, opBoolean UNION, opChamfer	6	PASS	PASS

#	Feature name	Export	Key operations	Inputs	Static	Onshape
17	**Hex Nut**	`hexNut`	skRegularPolygon, opExtrude	4	PASS	PASS
18	**Wheel With Spokes**	`wheelWithSpokes`	opExtrude, opPattern, rotationAround, opBoolean UNION	8	PASS	PASS
19	**Picture Frame**	`pictureFrame`	opExtrude	5	PASS	PASS
20	**Sphere**	`sphereBall`	opRevolve	2	PASS	PASS
21	**Torus**	`torusRing`	opRevolve	3	PASS	n/a
22	**D-Shaft**	`dShaftKeyed`	opExtrude, opBoolean SUBTRACTION	4	PASS	n/a
23	**Knurled Knob**	`knurledKnob`	opExtrude, opPattern, rotationAround, opBoolean SUBTRACTION	6	PASS	n/a
24	**Wedge Ramp**	`wedgeRamp`	opExtrude	4	PASS	n/a

Static is the project's strict FeatureScript validator. Onshape means the feature was executed in a real Onshape Part Studio through the Onshape API and reported no errors.

3. Installing and using a feature in Onshape

- Open (or create) an Onshape document. Click + (Insert new tab) → Create Feature Studio.
- Delete the starter text and paste the full source of a feature from the appendix (or from `data/fs_examples/<file>.fs` in the repository).
- Click Commit. The Feature Studio should show no errors.
- Switch to a Part Studio. Click Custom features on the toolbar (the puzzle-piece icon) and add the feature to the toolbar.
- Click the feature, optionally pick a plane, adjust the parameters and click the green check mark. Double-click the feature in the Feature list at any time to change its dimensions.

To share a feature with others, make the document public (or share it), then they can add it from Custom features → Add custom features by searching for the document.

Generating a new feature from text

Run the web app (see Section 5), type a description and copy the returned FeatureScript into a Feature Studio exactly as above. From the command line:

```
npm install
npm run test:e2e -- "Create a coffee mug with a handle"
```

4. How the generator works

Stage	What happens
Dimension extraction	A fast model parses the prompt into a shape class and dimensions.
Retrieval	The prompt is matched against the feature library manifest, CAD knowledge rows, Omni-CAD caption summaries and indexed FeatureScript documentation. The closest validated feature is injected as a worked example.
Generate → test → fix	Up to 3 candidates. Each one is sanitized and checked by a strict FeatureScript 2931 validator, and blocking errors are fed into the next attempt. If Onshape API keys are configured, candidates are also compiled against a live Part Studio.
Repair and fallback	Failing candidates get an AI repair pass. If everything fails, a deterministic builder or the closest library feature is returned, so a request always produces usable FeatureScript.
DeepCAD path (optional)	A Text2CAD model (DeepCAD sketch-and-extrude format) predicts the exact sketch planes, curves and extrude depths. A deterministic compiler (`deepcadCompiler.js`) turns that into FeatureScript with one adjustable Overall Size parameter, so the model never writes code directly.

5. Running in the cloud

The app is a Node 22 Express server and runs on any cloud host. With no API keys it still serves FeatureScript from the validated library and deterministic builders. Adding `GROQ_API_KEY` enables AI generation.

Target	How
Render	New → Blueprint → select the repository. `render.yaml` configures build, start, the `/health` check and secrets.
Any container host	`docker build -t cad-ai .` then `docker run -p 10000:10000 -e GROQ_API_KEY=… cad-ai` (Railway, Fly.io, Cloud Run, a VPS).
Continuous integration	GitHub Actions runs the validator, plan-compiler and DeepCAD regressions and boots the server on every push.
DeepCAD model (optional)	Text2CAD on a free Hugging Face Space (`hf_space/`); set `DEEPCAD_MODEL_URL`.
GPU model (optional)	CADFS-2B on Modal (scales to zero); set `CADFS_MODEL_URL`.

6. Quality and validation

Check	Command	Result
Library features pass the strict validator	`node scripts/submission_examples.mjs`	24 / 24 pass
Validator regression (templates, builders, known-bad code)	`npm run test:validator`	all pass
Plan compiler regression	`npm run test:plan`	all pass
DeepCAD compiler regression	`npm run test:deepcad`	all pass
Library features executed in a live Onshape Part Studio	`node scripts/submission_examples.mjs` (with Onshape API keys)	20 / 20 pass

The strict validator is a static check. It catches the failure classes the project has seen in practice: unbalanced braces, unitless vectors, nonexistent operations, wrong query arguments, and parameters multiplied by units twice. The live check goes further: each feature's source is sent to a real Onshape Part Studio, executed by Onshape's own FeatureScript engine, and any error it reports counts as a failure.

FeatureScript conventions the library follows

- Patterns use `opPattern` with `rotationAround` / `transform` arrays (`opPatternCircular` and `opPatternLinear` do not exist).
- Strings concatenate with ~. Arrays grow with `append()` and have no `.length`.
- `isLength` parameters already carry units, so the body never multiplies them by `inch`.
- Holes are inner circles in the same sketch (`qSketchRegion(id, true)`) or an extrude plus `opBoolean` SUBTRACTION.
- Revolve axes are `Line` values, never queries. Every sketch is closed with `skSolve`.

Appendix A. Feature source code

The complete source of every feature, exactly as it ships in the repository. Each listing is a whole Feature Studio file.

A.1 Plate With Mounting Holes

File `plate_with_holes.fs` · export `plateWithMountingHoles` · parameters: Plane (Query), Width (Length), Height (Length), Thickness (Length), Hole Radius (Length), Hole Inset (Length)
Holes are cut by extruding the hole profile and boolean-subtracting it from the body.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Plate With Mounting Holes" }
export const plateWithMountingHoles = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Width" }
        isLength(definition.width, { (inch) : [0.5, 3.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Height" }
        isLength(definition.height, { (inch) : [0.5, 2.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Thickness" }
        isLength(definition.thickness, { (inch) : [0.05, 0.25, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Hole Radius" }
        isLength(definition.holeRadius, { (inch) : [0.02, 0.125, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Hole Inset" }
        isLength(definition.holeInset, { (inch) : [0.1, 0.4, 4.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var w = definition.width / inch;
        var h = definition.height / inch;

        var plateSketch = newSketchOnPlane(context, id + "plateSketch", { "sketchPlane" : skPlane });
        skRectangle(plateSketch, "plate", {
            "firstCorner" : vector(-w / 2, -h / 2) * inch,
            "secondCorner" : vector(w / 2, h / 2) * inch
        });
        skSolve(plateSketch);
        opExtrude(context, id + "plateBody", {
            "entities"  : qSketchRegion(id + "plateSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.thickness
        });

        // Holes: sketch the circles, extrude them, then SUBTRACT from the plate.
        var inset = definition.holeInset / inch;
        var holeSketch = newSketchOnPlane(context, id + "holeSketch", { "sketchPlane" : skPlane });
        skCircle(holeSketch, "leftHole", {
            "center" : vector(-w / 2 + inset, 0) * inch,
            "radius" : definition.holeRadius
        });
        skCircle(holeSketch, "rightHole", {
            "center" : vector(w / 2 - inset, 0) * inch,
            "radius" : definition.holeRadius
        });
        skSolve(holeSketch);
        opExtrude(context, id + "holeTools", {
            "entities"  : qSketchRegion(id + "holeSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.thickness
        });
        opBoolean(context, id + "cutHoles", {
            "tools" : qCreatedBy(id + "holeTools", EntityType.BODY),
            "targets" : qCreatedBy(id + "plateBody", EntityType.BODY),
            "operationType" : BooleanOperationType.SUBTRACTION
```

```
        });
    });
```

A.2 Cylinder With Bore

File `cylinder_with_bore.fs` · export `cylinderWithBore` · parameters: Plane (Query), Outer Radius (Length), Bore Radius (Length), Height (Length)
Concentric circles in one sketch extrude directly into a tube, no boolean needed.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Cylinder With Bore" }
export const cylinderWithBore = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Outer Radius" }
        isLength(definition.outerRadius, { (inch) : [0.1, 1.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Bore Radius" }
        isLength(definition.boreRadius, { (inch) : [0.02, 0.25, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Height" }
        isLength(definition.height, { (inch) : [0.1, 1.5, 24.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        // Concentric circles in ONE sketch: the region between them extrudes
        // into a tube, so no boolean subtraction is needed for the bore.
        var ringSketch = newSketchOnPlane(context, id + "ringSketch", { "sketchPlane" : skPlane });
        skCircle(ringSketch, "outer", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.outerRadius
        });
        skCircle(ringSketch, "inner", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.boreRadius
        });
        skSolve(ringSketch);

        opExtrude(context, id + "tubeBody", {
            "entities"  : qSketchRegion(id + "ringSketch", true),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.height
        });
    });
```

A.3 Open Top Box

File `open_box_shell.fs` · export `openTopBox` · parameters: Plane (Query), Width (Length), Depth (Length), Height (Length), Wall Thickness (Length)
Shell a solid with a negative thickness and the end cap face to open the top.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Open Top Box" }
export const openTopBox = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Width" }
        isLength(definition.width, { (inch) : [0.5, 4.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Depth" }
        isLength(definition.depth, { (inch) : [0.5, 3.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Height" }
        isLength(definition.height, { (inch) : [0.25, 2.0, 24.0] } as LengthBoundSpec);
```

```
        annotation { "Name" : "Wall Thickness" }
        isLength(definition.wallThickness, { (inch) : [0.02, 0.1, 1.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var w = definition.width / inch;
        var d = definition.depth / inch;

        var baseSketch = newSketchOnPlane(context, id + "baseSketch", { "sketchPlane" : skPlane });
        skRectangle(baseSketch, "base", {
            "firstCorner" : vector(-w / 2, -d / 2) * inch,
            "secondCorner" : vector(w / 2, d / 2) * inch
        });
        skSolve(baseSketch);
        opExtrude(context, id + "boxBody", {
            "entities"  : qSketchRegion(id + "baseSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.height
        });

        // Hollow the solid and open the top by shelling away the end cap face.
        opShell(context, id + "shellBox", {
            "entities"  : qCapEntity(id + "boxBody", CapType.END, EntityType.FACE),
            "thickness" : -definition.wallThickness
        });
    });
```

A.4 Cup With Handle

File `cup_with_handle.fs` · export `cupWithHandle` · parameters: Plane (Query), Cup Radius (Length), Cup Height (Length), Wall Thickness (Length), Handle Radius (Length), Handle Thickness (Length)
Shell the cup from its top cap, build the handle as a ring on a perpendicular plane, union it on.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Cup With Handle" }
export const cupWithHandle = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Cup Radius" }
        isLength(definition.cupRadius, { (inch) : [0.5, 1.5, 8.0] } as LengthBoundSpec);

        annotation { "Name" : "Cup Height" }
        isLength(definition.cupHeight, { (inch) : [0.5, 3.5, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Wall Thickness" }
        isLength(definition.wallThickness, { (inch) : [0.03, 0.12, 0.5] } as LengthBoundSpec);

        annotation { "Name" : "Handle Radius" }
        isLength(definition.handleRadius, { (inch) : [0.1, 0.8, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Handle Thickness" }
        isLength(definition.handleThickness, { (inch) : [0.05, 0.2, 1.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        // Cup wall: solid cylinder, then shell it open from the top cap.
        var cupSketch = newSketchOnPlane(context, id + "cupSketch", { "sketchPlane" : skPlane });
        skCircle(cupSketch, "cupOuter", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.cupRadius
        });
        skSolve(cupSketch);
        opExtrude(context, id + "cupBody", {
            "entities"  : qSketchRegion(id + "cupSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.cupHeight
        });
```

```
        opShell(context, id + "cupShell", {
            "entities"  : qCapEntity(id + "cupBody", CapType.END, EntityType.FACE),
            "thickness" : -definition.wallThickness
        });

        // Handle: a ring profile beside the cup on a vertical plane,
        // extruded thin and unioned onto the wall.
        var r = definition.cupRadius / inch;
        var hh = definition.cupHeight / inch;
        var hr = definition.handleRadius / inch;
        var handlePlane = plane(skPlane.origin + skPlane.normal * (hh / 2 * inch), skPlane.x);
        var handleSketch = newSketchOnPlane(context, id + "handleSketch", { "sketchPlane" : handlePlane });
        skCircle(handleSketch, "handleOuter", {
            "center" : vector(r + hr * 0.6, 0) * inch,
            "radius" : definition.handleRadius
        });
        skCircle(handleSketch, "handleInner", {
            "center" : vector(r + hr * 0.6, 0) * inch,
            "radius" : definition.handleRadius - definition.handleThickness
        });
        skSolve(handleSketch);
        opExtrude(context, id + "handleBody", {
            "entities"  : qSketchRegion(id + "handleSketch", true),
            "direction" : handlePlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.handleThickness
        });
        opBoolean(context, id + "joinHandle", {
            "tools" : qUnion([
                qCreatedBy(id + "handleBody", EntityType.BODY),
                qCreatedBy(id + "cupBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });
    });
```

A.5 Revolved Vase

File `revolved_vase.fs` · export `revolvedVase` · parameters: Plane (Query), Base Radius (Length), Belly Radius (Length), Neck Radius (Length), Height (Length)
Closed profile = axis line + spline + closing lines; the revolve axis is a Line value, never a query.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Revolved Vase" }
export const revolvedVase = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Base Radius" }
        isLength(definition.baseRadius, { (inch) : [0.25, 1.2, 8.0] } as LengthBoundSpec);

        annotation { "Name" : "Belly Radius" }
        isLength(definition.bellyRadius, { (inch) : [0.25, 1.8, 10.0] } as LengthBoundSpec);

        annotation { "Name" : "Neck Radius" }
        isLength(definition.neckRadius, { (inch) : [0.1, 0.6, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Height" }
        isLength(definition.height, { (inch) : [0.5, 5.0, 24.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var baseR = definition.baseRadius / inch;
        var bellyR = definition.bellyRadius / inch;
        var neckR = definition.neckRadius / inch;
        var ht = definition.height / inch;

        // Closed revolve profile: axis on x = 0, smooth spline on the outside,
        // straight lines closing the region back to the axis.
        var profileSketch = newSketchOnPlane(context, id + "profileSketch", { "sketchPlane" : skPlane });
        skLineSegment(profileSketch, "axisLine", {
            "start" : vector(0, 0) * inch,
            "end"   : vector(0, ht) * inch
```

```
        });
        skFitSpline(profileSketch, "outerProfile", { "points" : [
            vector(baseR, 0) * inch,
            vector(bellyR * 0.9, ht * 0.25) * inch,
            vector(bellyR, ht * 0.45) * inch,
            vector(bellyR * 0.75, ht * 0.7) * inch,
            vector(neckR, ht * 0.9) * inch,
            vector(neckR, ht) * inch
        ] });
        skLineSegment(profileSketch, "topClose", {
            "start" : vector(neckR, ht) * inch,
            "end"   : vector(0, ht) * inch
        });
        skLineSegment(profileSketch, "baseClose", {
            "start" : vector(0, 0) * inch,
            "end"   : vector(baseR, 0) * inch
        });
        skSolve(profileSketch);

        // The revolve axis must be a Line value, never a query.
        var revolveAxis = line(skPlane.origin, cross(skPlane.normal, skPlane.x));
        opRevolve(context, id + "vaseBody", {
            "entities"     : qSketchRegion(id + "profileSketch"),
            "axis"         : revolveAxis,
            "angleForward" : 2 * PI * radian
        });
    });
```

A.6 Square To Round Transition

File `lofted_transition.fs` · export `squareToRoundTransition` · parameters: Plane (Query), Base Width (Length), Top Radius (Length), Height (Length)
Each loft profile is its own solved sketch on its own plane; opLoft takes profileSubqueries.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Square To Round Transition" }
export const squareToRoundTransition = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Base Width" }
        isLength(definition.baseWidth, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Top Radius" }
        isLength(definition.topRadius, { (inch) : [0.1, 0.75, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Height" }
        isLength(definition.height, { (inch) : [0.25, 2.5, 18.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var w = definition.baseWidth / inch;

        // Profile 1: square on the base plane.
        var baseProfile = newSketchOnPlane(context, id + "baseProfile", { "sketchPlane" : skPlane });
        skRectangle(baseProfile, "square", {
            "firstCorner" : vector(-w / 2, -w / 2) * inch,
            "secondCorner" : vector(w / 2, w / 2) * inch
        });
        skSolve(baseProfile);

        // Profile 2: circle on a parallel plane offset by the height.
        var topPlane = plane(skPlane.origin + skPlane.normal * definition.height, skPlane.normal);
        var topProfile = newSketchOnPlane(context, id + "topProfile", { "sketchPlane" : topPlane });
        skCircle(topProfile, "round", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.topRadius
        });
        skSolve(topProfile);

        // opLoft in FS 2931 takes profileSubqueries, ordered along the path.
        opLoft(context, id + "transitionBody", {
            "profileSubqueries" : [qSketchRegion(id + "baseProfile"), qSketchRegion(id + "topProfile")]
```

```
        });
    });
```

A.7 Swept Curved Tube

File `swept_tube.fs` · export `sweptCurvedTube` · parameters: Plane (Query), Tube Radius (Length), Run Length (Length), Rise Height (Length)
Sweep needs an open path wire and a profile sketch on a plane perpendicular to the path start.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Swept Curved Tube" }
export const sweptCurvedTube = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Tube Radius" }
        isLength(definition.tubeRadius, { (inch) : [0.05, 0.25, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Run Length" }
        isLength(definition.runLength, { (inch) : [0.5, 4.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Rise Height" }
        isLength(definition.riseHeight, { (inch) : [0.1, 1.0, 12.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var run = definition.runLength / inch;
        var rise = definition.riseHeight / inch;

        // Path: one open spline wire on the base plane.
        var pathSketch = newSketchOnPlane(context, id + "pathSketch", { "sketchPlane" : skPlane });
        skFitSpline(pathSketch, "spine", { "points" : [
            vector(0, 0) * inch,
            vector(run * 0.35, rise) * inch,
            vector(run * 0.7, -rise * 0.5) * inch,
            vector(run, 0) * inch
        ] });
        skSolve(pathSketch);

        // Profile: circle on a plane perpendicular to the path start.
        var profilePlane = plane(skPlane.origin, skPlane.x);
        var profileSketch = newSketchOnPlane(context, id + "profileSketch", { "sketchPlane" : profilePlane });
        skCircle(profileSketch, "section", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.tubeRadius
        });
        skSolve(profileSketch);

        opSweep(context, id + "tubeBody", {
            "profiles" : qSketchRegion(id + "profileSketch"),
            "path"     : qCreatedBy(id + "pathSketch", EntityType.EDGE)
        });
    });
```

A.8 L Bracket With Fillet And Chamfer

File `l_bracket_fillet_chamfer.fs` · export `lBracketFilletChamfer` · parameters: Plane (Query), Leg Length (Length), Leg Height (Length), Leg Thickness (Length), Bracket Width (Length), Fillet Radius (Length)
Closed polyline profile + one extrude; fillet edges via qEdgeTopologyFilter over the body edges.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "L Bracket With Fillet And Chamfer" }
export const lBracketFilletChamfer = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Leg Length" }
```

```
        isLength(definition.legLength, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Leg Height" }
        isLength(definition.legHeight, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Leg Thickness" }
        isLength(definition.legThickness, { (inch) : [0.05, 0.25, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Bracket Width" }
        isLength(definition.bracketWidth, { (inch) : [0.25, 1.5, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Fillet Radius" }
        isLength(definition.filletRadius, { (inch) : [0.01, 0.06, 0.5] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var legL = definition.legLength / inch;
        var legH = definition.legHeight / inch;
        var t = definition.legThickness / inch;

        // Closed L-shaped polyline profile, then one extrude for the whole bracket.
        var lSketch = newSketchOnPlane(context, id + "lSketch", { "sketchPlane" : skPlane });
        skLineSegment(lSketch, "seg1", { "start" : vector(0, 0) * inch,      "end" : vector(legL, 0) * inch });
        skLineSegment(lSketch, "seg2", { "start" : vector(legL, 0) * inch,   "end" : vector(legL, t) * inch });
        skLineSegment(lSketch, "seg3", { "start" : vector(legL, t) * inch,   "end" : vector(t, t) * inch });
        skLineSegment(lSketch, "seg4", { "start" : vector(t, t) * inch,      "end" : vector(t, legH) * inch });
        skLineSegment(lSketch, "seg5", { "start" : vector(t, legH) * inch,   "end" : vector(0, legH) * inch });
        skLineSegment(lSketch, "seg6", { "start" : vector(0, legH) * inch,   "end" : vector(0, 0) * inch });
        skSolve(lSketch);

        opExtrude(context, id + "bracketBody", {
            "entities"  : qSketchRegion(id + "lSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.bracketWidth
        });

        // Soften every two-sided edge of the bracket body.
        opFillet(context, id + "softenEdges", {
            "entities" : qEdgeTopologyFilter(qOwnedByBody(qCreatedBy(id + "bracketBody", EntityType.BODY), EntityType.
    EDGE), EdgeTopology.TWO_SIDED),
            "radius"   : definition.filletRadius
        });
    });
```

A.9 Bolt Circle Flange

File `bolt_circle_flange.fs` · export `boltCircleFlange` · parameters: Plane (Query), Flange Radius (Length), Center Bore Radius (Length), Bolt Hole Radius (Length), Bolt Circle Radius (Length), Bolt Count (Integer), Thickness (Length)
Loop with var + trig places bolt holes in the SAME sketch; string ids use the ~ operator.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Bolt Circle Flange" }
export const boltCircleFlange = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Flange Radius" }
        isLength(definition.flangeRadius, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Center Bore Radius" }
        isLength(definition.boreRadius, { (inch) : [0.05, 0.5, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Bolt Hole Radius" }
        isLength(definition.boltHoleRadius, { (inch) : [0.02, 0.125, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Bolt Circle Radius" }
        isLength(definition.boltCircleRadius, { (inch) : [0.25, 1.4, 10.0] } as LengthBoundSpec);

        annotation { "Name" : "Bolt Count" }
        isInteger(definition.boltCount, { (unitless) : [2, 6, 24] } as IntegerBoundSpec);

        annotation { "Name" : "Thickness" }
```

```
        isLength(definition.thickness, { (inch) : [0.05, 0.375, 3.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        // Disc, center bore, and every bolt hole live in ONE sketch, so a single
        // extrude produces the finished flange with all holes already cut.
        var bcr = definition.boltCircleRadius / inch;
        var flangeSketch = newSketchOnPlane(context, id + "flangeSketch", { "sketchPlane" : skPlane });
        skCircle(flangeSketch, "rim", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.flangeRadius
        });
        skCircle(flangeSketch, "centerBore", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.boreRadius
        });
        for (var i = 0; i < definition.boltCount; i += 1)
        {
            // String entity ids are built with the ~ concatenation operator.
            var boltAngle = (i * 2 * PI / definition.boltCount) * radian;
            var bx = cos(boltAngle) * bcr;
            var by = sin(boltAngle) * bcr;
            skCircle(flangeSketch, "bolt" ~ i, {
                "center" : vector(bx, by) * inch,
                "radius" : definition.boltHoleRadius
            });
        }
        skSolve(flangeSketch);

        opExtrude(context, id + "flangeBody", {
            "entities"  : qSketchRegion(id + "flangeSketch", true),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.thickness
        });
    });
```

A.10 Base With Corner Posts

File `multi_body_union.fs` · export `baseWithCornerPosts` · parameters: Plane (Query), Base Width (Length), Base Depth (Length), Base Thickness (Length), Post Radius (Length), Post Height (Length)
Build each component as its own extrude, position via offset planes, union into one body.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Base With Corner Posts" }
export const baseWithCornerPosts = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Base Width" }
        isLength(definition.baseWidth, { (inch) : [1.0, 4.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Base Depth" }
        isLength(definition.baseDepth, { (inch) : [1.0, 3.0, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Base Thickness" }
        isLength(definition.baseThickness, { (inch) : [0.05, 0.25, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Post Radius" }
        isLength(definition.postRadius, { (inch) : [0.05, 0.2, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Post Height" }
        isLength(definition.postHeight, { (inch) : [0.25, 1.0, 12.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var w = definition.baseWidth / inch;
        var d = definition.baseDepth / inch;
        var pr = definition.postRadius / inch;
        // Base plate.
```

```
        var baseSketch = newSketchOnPlane(context, id + "baseSketch", { "sketchPlane" : skPlane });
        skRectangle(baseSketch, "base", {
            "firstCorner" : vector(-w / 2, -d / 2) * inch,
            "secondCorner" : vector(w / 2, d / 2) * inch
        });
        skSolve(baseSketch);
        opExtrude(context, id + "baseBody", {
            "entities"  : qSketchRegion(id + "baseSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.baseThickness
        });

        // Four corner posts in one sketch on the base top, extruded together,
        // then unioned with the base so the result is one solid body.
        var inset = pr * 2;
        var postPlane = plane(skPlane.origin + skPlane.normal * definition.baseThickness, skPlane.normal);
        var postSketch = newSketchOnPlane(context, id + "postSketch", { "sketchPlane" : postPlane });
        skCircle(postSketch, "post0", { "center" : vector(-w / 2 + inset, -d / 2 + inset) * inch, "radius" :
    definition.postRadius });
        skCircle(postSketch, "post1", { "center" : vector(w / 2 - inset, -d / 2 + inset) * inch, "radius" :
    definition.postRadius });
        skCircle(postSketch, "post2", { "center" : vector(w / 2 - inset, d / 2 - inset) * inch, "radius" : definition.
    postRadius });
        skCircle(postSketch, "post3", { "center" : vector(-w / 2 + inset, d / 2 - inset) * inch, "radius" :
    definition.postRadius });
        skSolve(postSketch);
        opExtrude(context, id + "postBodies", {
            "entities"  : qSketchRegion(id + "postSketch"),
            "direction" : postPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.postHeight
        });

        opBoolean(context, id + "unionAll", {
            "tools" : qUnion([
                qCreatedBy(id + "postBodies", EntityType.BODY),
                qCreatedBy(id + "baseBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });
    });
```

A.11 Stepped Shaft With Chamfer

File `chamfered_shaft.fs` · export `steppedShaftWithChamfer` · parameters: Plane (Query), Major Radius (Length), Major Length (Length), Minor Radius (Length), Minor Length (Length), Chamfer Width (Length)
Stack sections on offset planes, union them, then chamfer the body edges.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Stepped Shaft With Chamfer" }
export const steppedShaftWithChamfer = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Major Radius" }
        isLength(definition.majorRadius, { (inch) : [0.1, 0.5, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Major Length" }
        isLength(definition.majorLength, { (inch) : [0.25, 1.5, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Minor Radius" }
        isLength(definition.minorRadius, { (inch) : [0.05, 0.3, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Minor Length" }
        isLength(definition.minorLength, { (inch) : [0.25, 1.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Chamfer Width" }
        isLength(definition.chamferWidth, { (inch) : [0.005, 0.03, 0.25] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        // Large diameter section.
```

```
        var majorSketch = newSketchOnPlane(context, id + "majorSketch", { "sketchPlane" : skPlane });
        skCircle(majorSketch, "major", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.majorRadius
        });
        skSolve(majorSketch);
        opExtrude(context, id + "majorBody", {
            "entities"  : qSketchRegion(id + "majorSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.majorLength
        });

        // Smaller diameter section stacked on top, then unioned into one shaft.
        var stepPlane = plane(skPlane.origin + skPlane.normal * definition.majorLength, skPlane.normal);
        var minorSketch = newSketchOnPlane(context, id + "minorSketch", { "sketchPlane" : stepPlane });
        skCircle(minorSketch, "minor", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.minorRadius
        });
        skSolve(minorSketch);
        opExtrude(context, id + "minorBody", {
            "entities"  : qSketchRegion(id + "minorSketch"),
            "direction" : stepPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.minorLength
        });
        // Break the sharp end edges with a chamfer.
        opChamfer(context, id + "endChamfer", {
            "entities"    : qEdgeTopologyFilter(qOwnedByBody(qCreatedBy(id + "minorBody", EntityType.BODY),
    EntityType.EDGE), EdgeTopology.TWO_SIDED),
            "chamferType" : ChamferType.EQUAL_OFFSETS,
            "width"       : definition.chamferWidth
        });
        opBoolean(context, id + "unionShaft", {
            "tools" : qUnion([
                qCreatedBy(id + "minorBody", EntityType.BODY),
                qCreatedBy(id + "majorBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });

    });
```

A.12 Disc With Patterned Bosses

File `circular_pattern_bosses.fs` · export `discWithPatternedBosses` · parameters: Plane (Query), Disc Radius (Length), Disc Thickness (Length), Boss Radius (Length), Boss Height (Length), Boss Circle Radius (Length), Instance Count (Integer)
Circular patterns use opPattern with rotationAround transforms and instanceNames (opPatternCircular does NOT exist); the original body is preserved so loop from 1.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Disc With Patterned Bosses" }
export const discWithPatternedBosses = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Disc Radius" }
        isLength(definition.discRadius, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Disc Thickness" }
        isLength(definition.discThickness, { (inch) : [0.05, 0.25, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Boss Radius" }
        isLength(definition.bossRadius, { (inch) : [0.05, 0.2, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Boss Height" }
        isLength(definition.bossHeight, { (inch) : [0.05, 0.3, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Boss Circle Radius" }
        isLength(definition.bossCircleRadius, { (inch) : [0.25, 1.4, 10.0] } as LengthBoundSpec);

        annotation { "Name" : "Instance Count" }
        isInteger(definition.instanceCount, { (unitless) : [2, 6, 36] } as IntegerBoundSpec);
    }
    {
```

```
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        // Base disc.
        var discSketch = newSketchOnPlane(context, id + "discSketch", { "sketchPlane" : skPlane });
        skCircle(discSketch, "disc", { "center" : vector(0, 0) * inch, "radius" : definition.discRadius });
        skSolve(discSketch);
        opExtrude(context, id + "discBody", {
            "entities"  : qSketchRegion(id + "discSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.discThickness
        });

        // ONE boss on top of the disc at the pattern radius.
        var bcr = definition.bossCircleRadius / inch;
        var bossPlane = plane(skPlane.origin + skPlane.normal * definition.discThickness, skPlane.normal);
        var bossSketch = newSketchOnPlane(context, id + "bossSketch", { "sketchPlane" : bossPlane });
        skCircle(bossSketch, "boss", { "center" : vector(bcr, 0) * inch, "radius" : definition.bossRadius });
        skSolve(bossSketch);
        opExtrude(context, id + "bossBody", {
            "entities"  : qSketchRegion(id + "bossSketch"),
            "direction" : bossPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.bossHeight
        });

        // CIRCULAR PATTERN: opPattern with rotationAround transforms.
        // The original boss is preserved, so build count-1 transforms starting at i = 1.
        // (opPatternCircular does NOT exist in FeatureScript.)
        var patternAxis = line(skPlane.origin, skPlane.normal);
        var transforms = [];
        var instanceNames = [];
        for (var i = 1; i < definition.instanceCount; i += 1)
        {
            var stepAngle = (i * 2 * PI / definition.instanceCount) * radian;
            transforms = append(transforms, rotationAround(patternAxis, stepAngle));
            instanceNames = append(instanceNames, "boss" ~ i);
        }
        opPattern(context, id + "bossPattern", {
            "entities" : qCreatedBy(id + "bossBody", EntityType.BODY),
            "transforms" : transforms,
            "instanceNames" : instanceNames
        });

        // Union the disc, the original boss, and every patterned boss into one body.
        opBoolean(context, id + "unionAll", {
            "tools" : qUnion([
                qCreatedBy(id + "bossBody", EntityType.BODY),
                qCreatedBy(id + "bossPattern", EntityType.BODY),
                qCreatedBy(id + "discBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });
    });
```

A.13 Heatsink With Linear Fins

File `linear_pattern_fins.fs` · export `heatsinkWithLinearFins` · parameters: Plane (Query), Base Width (Length), Base Depth (Length), Base Thickness (Length), Fin Thickness (Length), Fin Height (Length), Fin Count (Integer), Fin Spacing (Length)
Linear patterns use opPattern with transform(direction * spacing * i) translations and instanceNames (opPatternLinear does NOT exist).

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Heatsink With Linear Fins" }
export const heatsinkWithLinearFins = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Base Width" }
        isLength(definition.baseWidth, { (inch) : [0.5, 3.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Base Depth" }
        isLength(definition.baseDepth, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Base Thickness" }
```

```
        isLength(definition.baseThickness, { (inch) : [0.05, 0.2, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Fin Thickness" }
        isLength(definition.finThickness, { (inch) : [0.02, 0.08, 0.5] } as LengthBoundSpec);

        annotation { "Name" : "Fin Height" }
        isLength(definition.finHeight, { (inch) : [0.1, 0.6, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Fin Count" }
        isInteger(definition.finCount, { (unitless) : [2, 8, 40] } as IntegerBoundSpec);

        annotation { "Name" : "Fin Spacing" }
        isLength(definition.finSpacing, { (inch) : [0.05, 0.3, 2.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var w = definition.baseWidth / inch;
        var d = definition.baseDepth / inch;
        var ft = definition.finThickness / inch;

        // Base slab.
        var baseSketch = newSketchOnPlane(context, id + "baseSketch", { "sketchPlane" : skPlane });
        skRectangle(baseSketch, "base", {
            "firstCorner" : vector(-w / 2, -d / 2) * inch,
            "secondCorner" : vector(w / 2, d / 2) * inch
        });
        skSolve(baseSketch);
        opExtrude(context, id + "baseBody", {
            "entities"  : qSketchRegion(id + "baseSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.baseThickness
        });

        // ONE fin at the left edge of the base top.
        var finPlane = plane(skPlane.origin + skPlane.normal * definition.baseThickness, skPlane.normal);
        var finSketch = newSketchOnPlane(context, id + "finSketch", { "sketchPlane" : finPlane });
        skRectangle(finSketch, "fin", {
            "firstCorner" : vector(-w / 2, -d / 2) * inch,
            "secondCorner" : vector(-w / 2 + ft, d / 2) * inch
        });
        skSolve(finSketch);
        opExtrude(context, id + "finBody", {
            "entities"  : qSketchRegion(id + "finSketch"),
            "direction" : finPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.finHeight
        });

        // LINEAR PATTERN: opPattern with translation transforms along the width.
        // The original fin is preserved, so build count-1 transforms starting at i = 1.
        // (opPatternLinear does NOT exist in FeatureScript.)
        var transforms = [];
        var instanceNames = [];
        for (var i = 1; i < definition.finCount; i += 1)
        {
            transforms = append(transforms, transform(skPlane.x * (definition.finSpacing * i)));
            instanceNames = append(instanceNames, "fin" ~ i);
        }
        opPattern(context, id + "finPattern", {
            "entities" : qCreatedBy(id + "finBody", EntityType.BODY),
            "transforms" : transforms,
            "instanceNames" : instanceNames
        });

        // Union base, original fin, and all patterned fins.
        opBoolean(context, id + "unionAll", {
            "tools" : qUnion([
                qCreatedBy(id + "finBody", EntityType.BODY),
                qCreatedBy(id + "finPattern", EntityType.BODY),
                qCreatedBy(id + "baseBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });
    });
```

A.14 Pen With Tip And Clip

File `pen_with_tip_and_clip.fs` · export `penWithTipAndClip` · parameters: Plane (Query), Barrel Radius (Length), Barrel Length (Length), Tip Length (Length), Clip Length (Length), Clip Width (Length)
Everyday objects decompose into recognizable components: a pen is a barrel cylinder + lofted tapered tip + pocket clip, never one stretched primitive.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Pen With Tip And Clip" }
export const penWithTipAndClip = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Barrel Radius" }
        isLength(definition.barrelRadius, { (inch) : [0.1, 0.22, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Barrel Length" }
        isLength(definition.barrelLength, { (inch) : [1.0, 4.5, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Tip Length" }
        isLength(definition.tipLength, { (inch) : [0.2, 0.8, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Clip Length" }
        isLength(definition.clipLength, { (inch) : [0.3, 1.2, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Clip Width" }
        isLength(definition.clipWidth, { (inch) : [0.05, 0.15, 0.5] } as LengthBoundSpec);
    }
    {
        // A pen is NOT one stretched primitive. Decompose it into its
        // recognizable components: barrel + tapered writing tip + pocket clip.
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var r = definition.barrelRadius / inch;

        // 1. Barrel: cylinder along the plane normal.
        var barrelSketch = newSketchOnPlane(context, id + "barrelSketch", { "sketchPlane" : skPlane });
        skCircle(barrelSketch, "barrel", { "center" : vector(0, 0) * inch, "radius" : definition.barrelRadius });
        skSolve(barrelSketch);
        opExtrude(context, id + "barrelBody", {
            "entities"  : qSketchRegion(id + "barrelSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.barrelLength
        });

        // 2. Tapered tip: loft from the barrel end circle down to a small point circle.
        var tipStartPlane = plane(skPlane.origin + skPlane.normal * definition.barrelLength, skPlane.normal);
        var tipStartSketch = newSketchOnPlane(context, id + "tipStartSketch", { "sketchPlane" : tipStartPlane });
        skCircle(tipStartSketch, "tipBase", { "center" : vector(0, 0) * inch, "radius" : definition.barrelRadius });
        skSolve(tipStartSketch);

        var tipEndPlane = plane(skPlane.origin + skPlane.normal * (definition.barrelLength + definition.tipLength),
    skPlane.normal);
        var tipEndSketch = newSketchOnPlane(context, id + "tipEndSketch", { "sketchPlane" : tipEndPlane });
        skCircle(tipEndSketch, "tipPoint", { "center" : vector(0, 0) * inch, "radius" : definition.barrelRadius * 0.
    12 });
        skSolve(tipEndSketch);

        opLoft(context, id + "tipBody", {
            "profileSubqueries" : [qSketchRegion(id + "tipStartSketch"), qSketchRegion(id + "tipEndSketch")]
        });

        // 3. Pocket clip: thin bar lying along the barrel side near the top end.
        var cw = definition.clipWidth / inch;
        var clipSketch = newSketchOnPlane(context, id + "clipSketch", { "sketchPlane" : skPlane });
        skRectangle(clipSketch, "clip", {
            "firstCorner" : vector(r, -cw / 2) * inch,
            "secondCorner" : vector(r + cw, cw / 2) * inch
        });
        skSolve(clipSketch);
        opExtrude(context, id + "clipBody", {
            "entities"  : qSketchRegion(id + "clipSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
```

```
            "endDepth"  : definition.clipLength
        });

        // Union all pen components into one body.
        opBoolean(context, id + "unionPen", {
            "tools" : qUnion([
                qCreatedBy(id + "tipBody", EntityType.BODY),
                qCreatedBy(id + "clipBody", EntityType.BODY),
                qCreatedBy(id + "barrelBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });
    });
```

A.15 Bottle With Neck

File `bottle_with_neck.fs` · export `bottleWithNeck` · parameters: Plane (Query), Body Radius (Length), Body Height (Length), Neck Radius (Length), Neck Height (Length), Wall Thickness (Length)
A bottle is a revolved profile with belly, shoulder, and narrower neck, then shelled hollow through the neck opening; never a plain cylinder.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Bottle With Neck" }
export const bottleWithNeck = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Body Radius" }
        isLength(definition.bodyRadius, { (inch) : [0.5, 1.25, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Body Height" }
        isLength(definition.bodyHeight, { (inch) : [1.0, 4.0, 18.0] } as LengthBoundSpec);

        annotation { "Name" : "Neck Radius" }
        isLength(definition.neckRadius, { (inch) : [0.15, 0.45, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Neck Height" }
        isLength(definition.neckHeight, { (inch) : [0.25, 1.2, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Wall Thickness" }
        isLength(definition.wallThickness, { (inch) : [0.02, 0.08, 0.5] } as LengthBoundSpec);
    }
    {
        // A bottle is a revolved profile with a belly, a curved shoulder, and a
        // narrower neck, never a plain cylinder.
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var bodyR = definition.bodyRadius / inch;
        var bodyH = definition.bodyHeight / inch;
        var neckR = definition.neckRadius / inch;
        var neckH = definition.neckHeight / inch;
        var totalH = bodyH + neckH;

        var profileSketch = newSketchOnPlane(context, id + "profileSketch", { "sketchPlane" : skPlane });
        skLineSegment(profileSketch, "axisLine", {
            "start" : vector(0, 0) * inch,
            "end"   : vector(0, totalH) * inch
        });
        skFitSpline(profileSketch, "outerProfile", { "points" : [
            vector(bodyR, 0) * inch,
            vector(bodyR, bodyH * 0.55) * inch,
            vector(bodyR * 0.92, bodyH * 0.8) * inch,
            vector(neckR * 1.4, bodyH) * inch,
            vector(neckR, bodyH + neckH * 0.35) * inch,
            vector(neckR, totalH) * inch
        ] });
        skLineSegment(profileSketch, "topClose", {
            "start" : vector(neckR, totalH) * inch,
            "end"   : vector(0, totalH) * inch
        });
        skLineSegment(profileSketch, "baseClose", {
            "start" : vector(0, 0) * inch,
            "end"   : vector(bodyR, 0) * inch
        });
        skSolve(profileSketch);
```

```
        var revolveAxis = line(skPlane.origin, cross(skPlane.normal, skPlane.x));
        opRevolve(context, id + "bottleBody", {
            "entities"     : qSketchRegion(id + "profileSketch"),
            "axis"         : revolveAxis,
            "angleForward" : 2 * PI * radian
        });

        // Hollow the bottle by subtracting an inner revolve inset by the wall
        // thickness. (qCapEntity/opShell only work on extruded bodies, not revolves;
        // the tool profile extends past the top so the neck mouth opens cleanly.)
        var wt = definition.wallThickness / inch;
        var innerSketch = newSketchOnPlane(context, id + "innerSketch", { "sketchPlane" : skPlane });
        skLineSegment(innerSketch, "innerAxis", {
            "start" : vector(0, wt) * inch,
            "end"   : vector(0, totalH + wt) * inch
        });
        skFitSpline(innerSketch, "innerProfile", { "points" : [
            vector(bodyR - wt, wt) * inch,
            vector(bodyR - wt, bodyH * 0.55) * inch,
            vector(bodyR * 0.92 - wt, bodyH * 0.8) * inch,
            vector(neckR * 1.4 - wt, bodyH) * inch,
            vector(neckR - wt, bodyH + neckH * 0.35) * inch,
            vector(neckR - wt, totalH + wt) * inch
        ] });
        skLineSegment(innerSketch, "innerTopClose", {
            "start" : vector(neckR - wt, totalH + wt) * inch,
            "end"   : vector(0, totalH + wt) * inch
        });
        skLineSegment(innerSketch, "innerBaseClose", {
            "start" : vector(0, wt) * inch,
            "end"   : vector(bodyR - wt, wt) * inch
        });
        skSolve(innerSketch);
        opRevolve(context, id + "innerBody", {
            "entities"     : qSketchRegion(id + "innerSketch"),
            "axis"         : revolveAxis,
            "angleForward" : 2 * PI * radian
        });
        opBoolean(context, id + "hollowBottle", {
            "tools" : qCreatedBy(id + "innerBody", EntityType.BODY),
            "targets" : qCreatedBy(id + "bottleBody", EntityType.BODY),
            "operationType" : BooleanOperationType.SUBTRACTION
        });
    });
```

A.16 Hex Bolt

File `hex_bolt.fs` · export `hexBolt` · parameters: Plane (Query), Head Width (Length), Head Height (Length), Shaft Radius (Length), Shaft Length (Length), Chamfer Width (Length)
A bolt = hex polygon head + cylindrical shaft on an offset plane + union + tip chamfer.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Hex Bolt" }
export const hexBolt = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Head Width" }
        isLength(definition.headWidth, { (inch) : [0.1, 0.44, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Head Height" }
        isLength(definition.headHeight, { (inch) : [0.05, 0.17, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Shaft Radius" }
        isLength(definition.shaftRadius, { (inch) : [0.04, 0.125, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Shaft Length" }
        isLength(definition.shaftLength, { (inch) : [0.2, 1.0, 6.0] } as LengthBoundSpec);

        annotation { "Name" : "Chamfer Width" }
        isLength(definition.chamferWidth, { (inch) : [0.005, 0.02, 0.1] } as LengthBoundSpec);
    }
    {
        // A bolt = hex head + cylindrical shaft + chamfered tip, unioned.
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
```

```
            : evPlane(context, { "face" : definition.location });

        var hw = definition.headWidth / inch;

        // 1. Hex head: regular polygon profile extruded.
        var headSketch = newSketchOnPlane(context, id + "headSketch", { "sketchPlane" : skPlane });
        skRegularPolygon(headSketch, "hexHead", {
            "center" : vector(0, 0) * inch,
            "firstVertex" : vector(hw / 2, 0) * inch,
            "sides" : 6
        });
        skSolve(headSketch);
        opExtrude(context, id + "headBody", {
            "entities"  : qSketchRegion(id + "headSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.headHeight
        });

        // 2. Shaft: cylinder below the head.
        var shaftPlane = plane(skPlane.origin + skPlane.normal * definition.headHeight, skPlane.normal);
        var shaftSketch = newSketchOnPlane(context, id + "shaftSketch", { "sketchPlane" : shaftPlane });
        skCircle(shaftSketch, "shaft", { "center" : vector(0, 0) * inch, "radius" : definition.shaftRadius });
        skSolve(shaftSketch);
        opExtrude(context, id + "shaftBody", {
            "entities"  : qSketchRegion(id + "shaftSketch"),
            "direction" : shaftPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.shaftLength
        });

        // 3. Chamfer the shaft end so threads could start cleanly.
        opChamfer(context, id + "tipChamfer", {
            "entities"    : qEdgeTopologyFilter(qOwnedByBody(qCreatedBy(id + "shaftBody", EntityType.BODY),
    EntityType.EDGE), EdgeTopology.TWO_SIDED),
            "chamferType" : ChamferType.EQUAL_OFFSETS,
            "width"       : definition.chamferWidth
        });
        // 4. Union head and shaft into one bolt body.
        opBoolean(context, id + "unionBolt", {
            "tools" : qUnion([
                qCreatedBy(id + "shaftBody", EntityType.BODY),
                qCreatedBy(id + "headBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });

    });
```

A.17 Hex Nut

File `hex_nut.fs` · export `hexNut` · parameters: Plane (Query), Across Flats (Length), Thickness (Length), Bore Radius (Length)
A nut is a hex polygon with a concentric bore circle in the SAME sketch, extruded once. It is never a bolt shaft with no head, and never a bolt-with-shaft template reused for a nut.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Hex Nut" }
export const hexNut = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Across Flats" }
        isLength(definition.acrossFlats, { (inch) : [0.1, 0.44, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Thickness" }
        isLength(definition.thickness, { (inch) : [0.02, 0.17, 1.0] } as LengthBoundSpec);

        annotation { "Name" : "Bore Radius" }
        isLength(definition.boreRadius, { (inch) : [0.02, 0.1, 1.0] } as LengthBoundSpec);
    }
    {
        // A nut is a hex head with a concentric threaded bore, NOT a bolt: no shaft,
        // no chamfer, just the hex prism and its center hole in one sketch.
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });
```

```
        // "Across flats" is the distance between opposite flat sides; the circumscribed
        // radius (center to vertex) used by skRegularPolygon is acrossFlats / sqrt(3).
        var circumR = definition.acrossFlats / sqrt(3);

        var nutSketch = newSketchOnPlane(context, id + "nutSketch", { "sketchPlane" : skPlane });
        skRegularPolygon(nutSketch, "hex", {
            "center"      : vector(0, 0) * inch,
            "firstVertex" : vector(1, 0) * circumR,
            "sides"       : 6
        });
        skCircle(nutSketch, "bore", {
            "center" : vector(0, 0) * inch,
            "radius" : definition.boreRadius
        });
        skSolve(nutSketch);

        opExtrude(context, id + "nutBody", {
            "entities"  : qSketchRegion(id + "nutSketch", true),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.thickness
        });
    });
```

A.18 Wheel With Spokes

File `wheel_with_spokes.fs` · export `wheelWithSpokes` · parameters: Plane (Query), Rim Radius (Length), Rim Thickness (Length), Hub Radius (Length), Axle Radius (Length), Wheel Width (Length), Spoke Count (Integer), Spoke Width (Length)
A wheel = rim ring + hub ring with axle bore + one spoke circular-patterned via opPattern, all unioned.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Wheel With Spokes" }
export const wheelWithSpokes = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Rim Radius" }
        isLength(definition.rimRadius, { (inch) : [0.5, 2.0, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Rim Thickness" }
        isLength(definition.rimThickness, { (inch) : [0.05, 0.25, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Hub Radius" }
        isLength(definition.hubRadius, { (inch) : [0.1, 0.4, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Axle Radius" }
        isLength(definition.axleRadius, { (inch) : [0.02, 0.125, 2.0] } as LengthBoundSpec);

        annotation { "Name" : "Wheel Width" }
        isLength(definition.wheelWidth, { (inch) : [0.05, 0.4, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Spoke Count" }
        isInteger(definition.spokeCount, { (unitless) : [3, 6, 16] } as IntegerBoundSpec);

        annotation { "Name" : "Spoke Width" }
        isLength(definition.spokeWidth, { (inch) : [0.03, 0.15, 1.0] } as LengthBoundSpec);
    }
    {
        // A wheel = rim ring + center hub with axle bore + patterned spokes.
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var rimR = definition.rimRadius / inch;
        var rimT = definition.rimThickness / inch;
        var hubR = definition.hubRadius / inch;
        var sw = definition.spokeWidth / inch;

        // 1. Rim: ring from two concentric circles in one sketch.
        var rimSketch = newSketchOnPlane(context, id + "rimSketch", { "sketchPlane" : skPlane });
        skCircle(rimSketch, "rimOuter", { "center" : vector(0, 0) * inch, "radius" : definition.rimRadius });
        skCircle(rimSketch, "rimInner", { "center" : vector(0, 0) * inch, "radius" : definition.rimRadius -
    definition.rimThickness });
        skSolve(rimSketch);
        opExtrude(context, id + "rimBody", {
            "entities"  : qSketchRegion(id + "rimSketch", true),
```

```
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.wheelWidth
        });

        // 2. Hub: ring around the axle bore, same technique.
        var hubSketch = newSketchOnPlane(context, id + "hubSketch", { "sketchPlane" : skPlane });
        skCircle(hubSketch, "hubOuter", { "center" : vector(0, 0) * inch, "radius" : definition.hubRadius });
        skCircle(hubSketch, "axleBore", { "center" : vector(0, 0) * inch, "radius" : definition.axleRadius });
        skSolve(hubSketch);
        opExtrude(context, id + "hubBody", {
            "entities"  : qSketchRegion(id + "hubSketch", true),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.wheelWidth
        });

        // 3. One spoke: thin rectangle from hub to rim.
        var spokeSketch = newSketchOnPlane(context, id + "spokeSketch", { "sketchPlane" : skPlane });
        skRectangle(spokeSketch, "spoke", {
            "firstCorner" : vector(hubR * 0.8, -sw / 2) * inch,
            "secondCorner" : vector(rimR - rimT * 0.5, sw / 2) * inch
        });
        skSolve(spokeSketch);
        opExtrude(context, id + "spokeBody", {
            "entities"  : qSketchRegion(id + "spokeSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.wheelWidth
        });

        // 4. Circular pattern of the spoke (opPattern; opPatternCircular does NOT exist).
        var patternAxis = line(skPlane.origin, skPlane.normal);
        var transforms = [];
        var instanceNames = [];
        for (var i = 1; i < definition.spokeCount; i += 1)
        {
            transforms = append(transforms, rotationAround(patternAxis, (i * 2 * PI / definition.spokeCount) *
    radian));
            instanceNames = append(instanceNames, "spoke" ~ i);
        }
        opPattern(context, id + "spokePattern", {
            "entities" : qCreatedBy(id + "spokeBody", EntityType.BODY),
            "transforms" : transforms,
            "instanceNames" : instanceNames
        });

        // 5. Union rim, hub, and all spokes into one wheel body.
        opBoolean(context, id + "unionWheel", {
            "tools" : qUnion([
                qCreatedBy(id + "hubBody", EntityType.BODY),
                qCreatedBy(id + "spokeBody", EntityType.BODY),
                qCreatedBy(id + "spokePattern", EntityType.BODY),
                qCreatedBy(id + "rimBody", EntityType.BODY)
            ]),
            "operationType" : BooleanOperationType.UNION
        });
    });
```

A.19 Picture Frame

File `picture_frame.fs` · export `pictureFrame` · parameters: Plane (Query), Outer Width (Length), Outer Height (Length), Border Width (Length), Frame Depth (Length)

A frame is a rectangular ring: outer and inner rectangles in the SAME sketch extrude into the border with the middle open.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Picture Frame" }
export const pictureFrame = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Outer Width" }
        isLength(definition.outerWidth, { (inch) : [1.0, 6.0, 36.0] } as LengthBoundSpec);

        annotation { "Name" : "Outer Height" }
        isLength(definition.outerHeight, { (inch) : [1.0, 4.0, 36.0] } as LengthBoundSpec);
```

```
        annotation { "Name" : "Border Width" }
        isLength(definition.borderWidth, { (inch) : [0.1, 0.5, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Frame Depth" }
        isLength(definition.frameDepth, { (inch) : [0.05, 0.3, 2.0] } as LengthBoundSpec);
    }
    {
        // A frame is a rectangular RING: outer and inner rectangles in the SAME
        // sketch, so one extrude produces the border with the middle open.
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var w = definition.outerWidth / inch;
        var h = definition.outerHeight / inch;
        var b = definition.borderWidth / inch;

        var frameSketch = newSketchOnPlane(context, id + "frameSketch", { "sketchPlane" : skPlane });
        skRectangle(frameSketch, "outerRect", {
            "firstCorner" : vector(-w / 2, -h / 2) * inch,
            "secondCorner" : vector(w / 2, h / 2) * inch
        });
        skRectangle(frameSketch, "innerRect", {
            "firstCorner" : vector(-w / 2 + b, -h / 2 + b) * inch,
            "secondCorner" : vector(w / 2 - b, h / 2 - b) * inch
        });
        skSolve(frameSketch);

        opExtrude(context, id + "frameBody", {
            "entities"  : qSketchRegion(id + "frameSketch", true),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.frameDepth
        });
    });
```

A.20 Sphere

File `sphere_ball.fs` · export `sphereBall` · parameters: Plane (Query), Radius (Length)
A sphere is a semicircle arc plus a closing line drawn exactly on the axis, revolved 360 degrees; the closing line must be coincident with the revolve axis.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Sphere" }
export const sphereBall = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Radius" }
        isLength(definition.radius, { (inch) : [0.1, 1.0, 12.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var r = definition.radius / inch;

        // A sphere is a half-disk (semicircle arc + a closing line ON the axis)
        // revolved 360 degrees. The closing line must sit exactly on x = 0 so it
        // is coincident with the revolve axis.
        var profileSketch = newSketchOnPlane(context, id + "profileSketch", { "sketchPlane" : skPlane });
        skArc(profileSketch, "sphereArc", {
            "start" : vector(0, -r) * inch,
            "mid"   : vector(r, 0) * inch,
            "end"   : vector(0, r) * inch
        });
        skLineSegment(profileSketch, "axisClose", {
            "start" : vector(0, r) * inch,
            "end"   : vector(0, -r) * inch
        });
        skSolve(profileSketch);

        // The revolve axis must be a Line value, never a query.
        var revolveAxis = line(skPlane.origin, cross(skPlane.normal, skPlane.x));
        opRevolve(context, id + "sphereBody", {
```

```
            "entities"     : qSketchRegion(id + "profileSketch"),
            "axis"         : revolveAxis,
            "angleForward" : 2 * PI * radian
        });
    });
```

A.21 Torus

File `torus_ring.fs` · export `torusRing` · parameters: Plane (Query), Major Radius (Length), Tube Radius (Length)
A torus is a circle offset from (never touching) the revolve axis, revolved 360 degrees.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Torus" }
export const torusRing = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Major Radius" }
        isLength(definition.majorRadius, { (inch) : [0.3, 1.5, 12.0] } as LengthBoundSpec);

        annotation { "Name" : "Tube Radius" }
        isLength(definition.tubeRadius, { (inch) : [0.02, 0.3, 4.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var majorR = definition.majorRadius / inch;

        // A torus is a circle offset from the revolve axis (never touching it),
        // revolved 360 degrees around that axis.
        var profileSketch = newSketchOnPlane(context, id + "profileSketch", { "sketchPlane" : skPlane });
        skCircle(profileSketch, "tubeProfile", {
            "center" : vector(majorR, 0) * inch,
            "radius" : definition.tubeRadius
        });
        skSolve(profileSketch);

        // The revolve axis must be a Line value, never a query.
        var revolveAxis = line(skPlane.origin, cross(skPlane.normal, skPlane.x));
        opRevolve(context, id + "torusBody", {
            "entities"     : qSketchRegion(id + "profileSketch"),
            "axis"         : revolveAxis,
            "angleForward" : 2 * PI * radian
        });
    });
```

A.22 D-Shaft

File `d_shaft_keyed.fs` · export `dShaftKeyed` · parameters: Plane (Query), Shaft Radius (Length), Shaft Length (Length), Flat Depth (Length)
A D-shaft is a round shaft with an oversized rectangular tool boolean-subtracted past one side to leave a single flat face.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "D-Shaft" }
export const dShaftKeyed = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Shaft Radius" }
        isLength(definition.shaftRadius, { (inch) : [0.05, 0.375, 3.0] } as LengthBoundSpec);

        annotation { "Name" : "Shaft Length" }
        isLength(definition.shaftLength, { (inch) : [0.2, 1.5, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Flat Depth" }
        isLength(definition.flatDepth, { (inch) : [0.01, 0.08, 1.0] } as LengthBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
```

```
            : evPlane(context, { "face" : definition.location });

        var r = definition.shaftRadius / inch;
        var flatDepth = definition.flatDepth / inch;
        var flatX = r - flatDepth;

        // 1. Round shaft.
        var shaftSketch = newSketchOnPlane(context, id + "shaftSketch", { "sketchPlane" : skPlane });
        skCircle(shaftSketch, "shaft", { "center" : vector(0, 0) * inch, "radius" : definition.shaftRadius });
        skSolve(shaftSketch);
        opExtrude(context, id + "shaftBody", {
            "entities"  : qSketchRegion(id + "shaftSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.shaftLength
        });

        // 2. Flat-cutting tool: a block that overlaps one side of the shaft past
        // flatX, oversized in the cross direction to guarantee a clean cut.
        var flatSketch = newSketchOnPlane(context, id + "flatSketch", { "sketchPlane" : skPlane });
        skRectangle(flatSketch, "flatTool", {
            "firstCorner"  : vector(flatX, -(r + 0.2)) * inch,
            "secondCorner" : vector(r + 0.2, r + 0.2) * inch
        });
        skSolve(flatSketch);
        opExtrude(context, id + "flatTool", {
            "entities"  : qSketchRegion(id + "flatSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.shaftLength
        });

        // 3. Subtract the flat tool from the round shaft to leave one flat face.
        opBoolean(context, id + "cutFlat", {
            "tools"         : qCreatedBy(id + "flatTool", EntityType.BODY),
            "targets"       : qCreatedBy(id + "shaftBody", EntityType.BODY),
            "operationType" : BooleanOperationType.SUBTRACTION
        });
    });
```

A.23 Knurled Knob

File `knurled_knob.fs` · export `knurledKnob` · parameters: Plane (Query), Knob Radius (Length), Knob Height (Length), Groove Depth (Length), Groove Width (Length), Groove Count (Integer)
Knurling is a thin radial groove tool circular-patterned with opPattern around the knob axis, then boolean-subtracted from the base cylinder (not unioned, since it removes material).

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Knurled Knob" }
export const knurledKnob = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Knob Radius" }
        isLength(definition.knobRadius, { (inch) : [0.15, 0.5, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Knob Height" }
        isLength(definition.knobHeight, { (inch) : [0.1, 0.4, 4.0] } as LengthBoundSpec);

        annotation { "Name" : "Groove Depth" }
        isLength(definition.grooveDepth, { (inch) : [0.005, 0.03, 0.2] } as LengthBoundSpec);

        annotation { "Name" : "Groove Width" }
        isLength(definition.grooveWidth, { (inch) : [0.01, 0.05, 0.3] } as LengthBoundSpec);

        annotation { "Name" : "Groove Count" }
        isInteger(definition.grooveCount, { (unitless) : [6, 24, 60] } as IntegerBoundSpec);
    }
    {
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var knobR = definition.knobRadius / inch;
        var grooveDepth = definition.grooveDepth / inch;
        var grooveW = definition.grooveWidth / inch;
```

```
        // 1. Base knob cylinder.
        var knobSketch = newSketchOnPlane(context, id + "knobSketch", { "sketchPlane" : skPlane });
        skCircle(knobSketch, "knobOuter", { "center" : vector(0, 0) * inch, "radius" : definition.knobRadius });
        skSolve(knobSketch);
        opExtrude(context, id + "knobBody", {
            "entities"  : qSketchRegion(id + "knobSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.knobHeight
        });

        // 2. ONE groove-cutting tool: a thin radial slot at the rim, running the
        // full height, slightly oversized past the outer surface for a clean cut.
        var grooveSketch = newSketchOnPlane(context, id + "grooveSketch", { "sketchPlane" : skPlane });
        skRectangle(grooveSketch, "groove0", {
            "firstCorner"  : vector(knobR - grooveDepth, -grooveW / 2) * inch,
            "secondCorner" : vector(knobR + 0.1, grooveW / 2) * inch
        });
        skSolve(grooveSketch);
        opExtrude(context, id + "grooveTool", {
            "entities"  : qSketchRegion(id + "grooveSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.knobHeight
        });

        // 3. CIRCULAR PATTERN the groove tool around the knob axis.
        // The original groove is preserved, so build count-1 transforms from i = 1.
        // (opPatternCircular does NOT exist in FeatureScript.)
        var patternAxis = line(skPlane.origin, skPlane.normal);
        var transforms = [];
        var instanceNames = [];
        for (var i = 1; i < definition.grooveCount; i += 1)
        {
            var stepAngle = (i * 2 * PI / definition.grooveCount) * radian;
            transforms = append(transforms, rotationAround(patternAxis, stepAngle));
            instanceNames = append(instanceNames, "groove" ~ i);
        }
        opPattern(context, id + "groovePattern", {
            "entities" : qCreatedBy(id + "grooveTool", EntityType.BODY),
            "transforms" : transforms,
            "instanceNames" : instanceNames
        });

        // 4. Subtract the original groove and every patterned groove from the knob.
        opBoolean(context, id + "cutGrooves", {
            "tools" : qUnion([
                qCreatedBy(id + "grooveTool", EntityType.BODY),
                qCreatedBy(id + "groovePattern", EntityType.BODY)
            ]),
            "targets"       : qCreatedBy(id + "knobBody", EntityType.BODY),
            "operationType" : BooleanOperationType.SUBTRACTION
        });
    });
```

A.24 Wedge Ramp

File `wedge_ramp.fs` · export `wedgeRamp` · parameters: Plane (Query), Ramp Length (Length), Ramp Height (Length), Ramp Width (Length)
A ramp/wedge is a closed right-triangle profile (base, vertical back, sloped hypotenuse) extruded once.

```
FeatureScript 2931;
import(path : "onshape/std/geometry.fs", version : "2931.0");

annotation { "Feature Type Name" : "Wedge Ramp" }
export const wedgeRamp = defineFeature(function(context is Context, id is Id, definition is map)
    precondition
    {
        annotation { "Name" : "Plane", "Filter" : GeometryType.PLANE, "MaxNumberOfPicks" : 1 }
        definition.location is Query;

        annotation { "Name" : "Ramp Length" }
        isLength(definition.rampLength, { (inch) : [0.5, 4.0, 36.0] } as LengthBoundSpec);

        annotation { "Name" : "Ramp Height" }
        isLength(definition.rampHeight, { (inch) : [0.25, 1.5, 24.0] } as LengthBoundSpec);

        annotation { "Name" : "Ramp Width" }
        isLength(definition.rampWidth, { (inch) : [0.25, 2.0, 36.0] } as LengthBoundSpec);
    }
    {
```

```
        var skPlane = isQueryEmpty(context, definition.location)
            ? plane(WORLD_ORIGIN, Z_DIRECTION)
            : evPlane(context, { "face" : definition.location });

        var rampL = definition.rampLength / inch;
        var rampH = definition.rampHeight / inch;

        // Closed right-triangle profile (flat base, vertical back, sloped
        // hypotenuse), then one extrude for the whole wedge.
        var rampSketch = newSketchOnPlane(context, id + "rampSketch", { "sketchPlane" : skPlane });
        skLineSegment(rampSketch, "base", {
            "start" : vector(0, 0) * inch,
            "end"   : vector(rampL, 0) * inch
        });
        skLineSegment(rampSketch, "back", {
            "start" : vector(rampL, 0) * inch,
            "end"   : vector(rampL, rampH) * inch
        });
        skLineSegment(rampSketch, "slope", {
            "start" : vector(rampL, rampH) * inch,
            "end"   : vector(0, 0) * inch
        });
        skSolve(rampSketch);

        opExtrude(context, id + "rampBody", {
            "entities"  : qSketchRegion(id + "rampSketch"),
            "direction" : skPlane.normal,
            "endBound"  : BoundingType.BLIND,
            "endDepth"  : definition.rampWidth
        });
    });
```